

Guide 4 — Front-end et interface

Ta mission : que l'application soit belle, claire et utilisable sur téléphone. Le design reste un critère du jury, et c'est ce qu'il voit en premier. Claude fournit la direction visuelle ; toi, tu l'intègres proprement.

1. Où tu travailles

resources/views/welcome.blade.php	← PAGE D'ACCUEIL (fonctionnalité de base dans tous les sujets)
resources/views/dashboard.blade.php	← tableau de bord après connexion
resources/views/pages/<fonctionnalité>/	← pages générées (index, form, show)
resources/views/layouts/app/sidebar.blade.php	← menu de gauche
resources/css/app.css	← couleurs et thème
app/Providers/Filament/AdminPanelProvider.php	← couleur de l'admin

Pendant que tu travailles : `composer run dev` doit tourner (la page se recharge toute seule).

2. Flux : les composants à connaître

La version gratuite suffit. Documentation : fluxui.dev (cocher « free »).

```
<flux:heading size="xl" level="1">Titre de page</flux:heading>
<flux:text>Paragraphe secondaire</flux:text>

<flux:button variant="primary" icon="plus" :href="route('signalements.create')"
wire:navigate>Ajouter</flux:button>
<flux:button variant="ghost" icon="pencil-square" size="sm" />
<flux:button variant="danger" wire:click="delete" wire:confirm="Supprimer
?">Supprimer</flux:button>

<flux:input wire:model="titre" label="Titre" required />
<flux:input wire:model.live.debounce.300ms="search" icon="magnifying-glass"
placeholder="Rechercher..." />
<flux:textarea wire:model="description" label="Description" rows="5" />
<flux:select wire:model="niveau" label="Niveau">
  <flux:select.option value="faible">Faible</flux:select.option>
</flux:select>
<flux:checkbox wire:model="ouvert" label="Ouvert" />

<flux:badge color="red" size="sm">Critique</flux:badge>
<flux:card>Contenu encadré</flux:card>
<flux:callout icon="information-circle">Message d'information</flux:callout>
<flux:separator />
```

Tableau paginé :

```
<flux:table :paginate="$this->items">
  <flux:table.columns>
    <flux:table.column>Titre</flux:table.column>
  </flux:table.columns>
  <flux:table.rows>
    @foreach ($this->items as $item)
      <flux:table.row wire:key="row-{{ $item->id }}">
        <flux:table.cell>{{ $item->titre }}</flux:table.cell>
      </flux:table.row>
    @endforeach
  </flux:table.rows>
</flux:table>
```

Fenêtre modale :

```
<flux:modal.trigger name="confirmer">
  <flux:button>Ouvrir</flux:button>
</flux:modal.trigger>

<flux:modal name="confirmer" class="md:w-96">
  <flux:heading>Confirmer ?</flux:heading>
  <flux:button wire:click="valider" variant="primary">Oui</flux:button>
</flux:modal>
```

Message après une action (dans le PHP du composant) : `Flux::toast(variant: 'success', text: 'Enregistré.');`

Icônes : noms Heroicons (heroicons.com), ex. `map-pin`, `bell`, `heart`, `chart-bar`, `exclamation-triangle`.

3. Livewire : les gestes du front

Besoin	Code
Lier un champ	<code>wire:model="titre"</code>
Filtre en direct	<code>wire:model.live="niveau"</code>
Recherche à la frappe	<code>wire:model.live.debounce.300ms="search"</code>
Bouton qui appelle PHP	<code>wire:click="toggleFavori({{ \$item->id }})"</code>
Envoyer un formulaire	<code><form wire:submit="save"></code>
Confirmation	<code>wire:confirm="Supprimer cet élément ?"</code>
Afficher pendant le chargement	<code><div wire:loading>Chargement...</div></code>
Désactiver pendant l'envoi	<code><flux:button type="submit" wire:loading.attr="disabled"></code>
Navigation sans rechargement	<code>wire:navigate</code> sur les liens
Erreur d'un champ	automatique avec <code>flux:input ; sinon @error('titre') {{ \$message }}</code> <code>@enderror</code>

4. Les 4 états de chaque écran

Le jury remarque immédiatement une page qui ne les gère pas :

```
@if ($this->items->isEmpty())
    <flux:card class="py-12 text-center">
        <flux:heading>Aucun signalement pour le moment</flux:heading>
        <flux:text class="mt-2">Soyez le premier à signaler un incident.</flux:text>
        <flux:button class="mt-4" variant="primary" :href="route('signalements.create')">
wire:navigate>Signaler</flux:button>
    </flux:card>
@else
    ...
@endif

<div wire:loading.delay class="text-sm text-zinc-500">Chargement...</div>
```

- **Vide** : un message utile + une action.
- **Chargement** : `wire:loading`.
- **Erreur** : messages de validation en français, clairs.
- **Succès** : `Flux::toast(...)`.

5. Tailwind : l'essentiel

Espacements	p-4 px-6 py-2 m-4 mt-2 gap-4 space-y-6
Mise en page	flex flex-col items-center justify-between grid grid-cols-1 md:grid-cols-3
Tailles	w-full max-w-2xl h-40 size-10
Texte	text-sm text-lg font-medium font-semibold text-zinc-500
Formes	rounded-lg rounded-xl border shadow-sm
Images	object-cover aspect-video
Sombre	dark:bg-zinc-900 dark:text-white
Responsive	sm: (≥640px) md: (≥768px) lg: (≥1024px)

Mobile d'abord : écrire les classes pour le téléphone, puis ajouter `md:` / `lg:` pour les grands écrans.

```
<div class="grid grid-cols-1 gap-4 md:grid-cols-2 lg:grid-cols-3">
```

Tester chaque écran en largeur téléphone : outils du navigateur (F12) → mode responsive.

6. L'identité visuelle du sujet

Dès que le sujet tombe, Claude propose une palette et une ambiance. Tu l'appliques à deux endroits :

Application — `resources/css/app.css` , dans le bloc `@theme` , les variables `--color-accent` , `--color-accent-content` , `--color-accent-foreground` :

```
@theme {
  --color-accent: var(--color-orange-500);
  --color-accent-content: var(--color-orange-600);
  --color-accent-foreground: var(--color-white);
}
```

Admin — `app/Providers/Filament/AdminPanelProvider.php` : `->colors(['primary' => Color::Orange])` , et `->brandName('Nom du projet')` .

Également : le nom de l'application (`APP_NAME` dans `.env`), le logo (`resources/views/components/app-logo.blade.php`), le titre du menu.

7. La page d'accueil (à soigner en premier)

C'est la fonctionnalité de base n°1 des trois sujets d'exemple, et la première chose que voit le jury. Structure efficace :

1. **Bandeau** : nom, phrase qui explique le service, bouton principal (« Commencer » → inscription, ou « Accéder » si connecté).
2. **3 blocs** : les 3 fonctionnalités principales, avec icône et une phrase.
3. **Chiffres** en direct (`Signalement::count()` ...) : montre que l'app est vivante.
4. **Pied de page** : équipe, mentions.

```
@auth
    <flux:button variant="primary" :href="route('dashboard')">Accéder à mon
espace</flux:button>
@else
    <flux:button variant="primary" :href="route('register')">Créer un compte</flux:button>
@endauth
```

8. Ajouter une entrée au menu

Le générateur le fait. À la main, dans `sidebar.blade.php`, **au-dessus** du marqueur `{{-- make:feature:nav --}}` :

```
<flux:sidebar.item icon="map" :href="route('carte')" :current="request()->routeIs('carte')"
wire:navigate>
    Carte
</flux:sidebar.item>
```

9. Une carte (Leaflet, sans installation)

Fonctionnalité fréquente (points de regroupement, signalements, animaux proches...). Dans une page Livewire :

```
#[Computed]
public function points(): array
{
    return Signalement::whereNotNull('latitude')->get(['id', 'titre', 'latitude', 'longitude'])
        ->map(fn ($s) => ['lat' => (float) $s->latitude, 'lng' => (float) $s->longitude,
'titre' => $s->titre])
        ->all();
}
```

```

@assets
  <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />
  <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
@endassets

<div wire:ignore
  x-data="{ points: @js($this->points) }"
  x-init="
    const map = L.map($el).setView([-18.91, 47.52], 13);
    L.tileLayer('https://tile.openstreetmap.org/{z}/{x}/{y}.png', { attribution: '©
OpenStreetMap' }).addTo(map);
    points.forEach(p => {
      const texte = document.createElement('span');
      texte.textContent = p.titre;          // texte brut : aucune injection HTML
possible
      L.marker([p.lat, p.lng]).addTo(map).bindPopup(texte);
    });
  "
  class="h-96 w-full rounded-xl"></div>

```

- `wire:ignore` : Livewire ne touche pas à la carte après son affichage.
- Coordonnées par défaut : Antananarivo. Adapter au sujet.
- Le titre passe par `textContent` : même s'il contient du HTML saisi par un utilisateur, il s'affiche comme du texte.

Géolocalisation (« autour de moi ») : `navigator.geolocation.getCurrentPosition(pos => ...)` dans le `x-init`, puis `$wire.set('lat', pos.coords.latitude)`. Autorisée par nos en-têtes (`geolocation=(self)`), et nécessite le HTTPS (c'est le cas en ligne).

10. Images

- Upload : déjà géré par le générateur (type `image`).
- Affichage : `titre }}" class="aspect-video w-full rounded-xl object-cover">`
- Toujours un `alt` (accessibilité), toujours `object-cover` (pas d'image déformée).
- Pas d'image lourde dans `public/` : le jury note aussi la performance.

11. Checklist avant de dire « terminé »

- ☐ Lisible et utilisable sur téléphone (menu, tableaux, formulaires)
- ☐ États vide / chargement / erreur / succès présents
- ☐ Textes en français, sans faute, cohérents avec le thème du sujet
- ☐ Boutons « Modifier » / « Supprimer » visibles seulement si autorisé (`@can`)
- ☐ Données de démo présentes : la page n'est jamais vide devant le jury
- ☐ Mode sombre lisible (si l'application l'affiche)